المذكرة الاحترافية

Software Engineering for Machine Learning Engineers

الفصل السابع: Testing

الطريقة الوحيدة اللي بتديك ثقة حقيقية إن الكود شغال صح، وهيفضل كده بعد أي تعديل مستقبلي

Unit Testing • Integration Testing • Mocking

جدول المحتويات

جدول المحتويات 2
مقدمة الفصل السابع: إزاي تتأكد إن الكود فعلًا شغال؟ 4
٧٫١ Unit Testing — اختبار الوحدات 5
١. ليه Unit Testing مهم؟ 5
٢. المشكلة اللي بيحلها Unit Testing 5
٣. إزاي كانت الشركات بتتعامل قبل تبني Unit Testing 5
٤. ليه الطريقة القديمة مش مناسبة 5
٥. الفكرة الأساسية وراء Unit Testing 5
٦. المفاهيم الأساسية 5
٧. المكونات 5
٨. إزاي المبدأ ده بيشتغل داخليًا 5
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 6
١٠. فين موقعه داخل مشروع ML حقيقي 6
١١. امتى تكتب Unit Tests شاملة 6
١٢. امتى ممكن تكتفي باختبارات محدودة 6
١٣. أفضل الممارسات 6
١٤. أشهر الأخطاء الشائعة 6
١٥. اعتبارات الأداء 6
١٦. الاعتبارات الأمنية 6
١٧. اعتبارات قابلية التوسع 7
١٨. مقارنة مع نهج بديل 7
١٩. مثال عملي صغير 7
٢٠. مثال من مشروع Production حقيقي 7
٢١. ملخص الموضوع 7
٧٫٢ Integration Testing — اختبار التكامل 8
١. ليه Integration Testing مهم؟ 8
٢. المشكلة اللي بيحلها Integration Testing 8
٣. إزاي كانت الشركات بتتعامل قبل تبني Integration Testing 8
٤. ليه الطريقة القديمة مش مناسبة 8
٥. الفكرة الأساسية وراء Integration Testing 8
٦. المفاهيم الأساسية 8
٧. المكونات 8
٨. إزاي المبدأ ده بيشتغل داخليًا 8
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 9
١٠. فين موقعه داخل مشروع ML حقيقي 9
١١. امتى تكتب Integration Tests 9
١٢. امتى Integration Testing يكون زيادة عن اللزوم 9
١٣. أفضل الممارسات 9
١٤. أشهر الأخطاء الشائعة 9
١٥. اعتبارات الأداء 9

١٦. الاعتبارات الأمنية 9
١٧. اعتبارات قابلية التوسع 10
١٨. مقارنة مع نهج بديل 10
١٩. مثال عملي صغير 10
٢٠. مثال من مشروع Production حقيقي 10
٢١. ملخص الموضوع 10
٧٫٣ Mocking — محاكاة الاعتماديات 11
١. ليه Mocking مهم؟ 11
٢. المشكلة اللي بيحلها Mocking 11
٣. إزاي كانت الشركات بتتعامل قبل تبني Mocking 11
٤. ليه الطريقة القديمة مش مناسبة 11
٥. الفكرة الأساسية وراء Mocking 11
٦. المفاهيم الأساسية 11
٧. المكونات 11
٨. إزاي المبدأ ده بيشتغل داخليًا 11
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 11
١٠. فين موقعه داخل مشروع ML حقيقي 12
١١. امتى تستخدم Mocking 12
١٢. امتى Mocking يكون مبالغ فيه 12
١٣. أفضل الممارسات 12
١٤. أشهر الأخطاء الشائعة 12
١٥. اعتبارات الأداء 12
١٦. الاعتبارات الأمنية 12
١٧. اعتبارات قابلية التوسع 13
١٨. مقارنة مع نهج بديل 13
١٩. مثال عملي صغير 13
٢٠. مثال من مشروع Production حقيقي 13
٢١. ملخص الموضوع 13
مشروع عملي على الفصل: بناء مجموعة اختبارات متكاملة لـ Mini ML Pipeline 14
خطوات التنفيذ 14
تحديات شائعة عند التطبيق 14
إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي 14
ملخص الفصل السابع وربطه بالفصل القادم 14

مقدمة الفصل السابع: إزاي تتأكد إن الكود فعلًا شغال؟

في الفصل اللي فات اتعلمنا إزاي نتعامل مع الأخطاء بشكل احترافي: Exceptions منظمة، Validation تمنع البيانات الفاسدة من الدخول، وLogging يوثق الأحداث المهمة. لكن السؤال اللي لسه من غير إجابة: إزاي تتأكد إن كل ده فعلًا شغال صح، مش بس نظريًا؟

الإجابة هي Testing. الاختبار هو الطريقة الوحيدة اللي بتديك ثقة حقيقية (مش إحساس) إن الكود بيعمل اللي المفروض يعمله، وإنه هيفضل يعمل كده حتى بعد أي تعديل مستقبلي. من غير اختبارات، أي تعديل في الكود بيبقى مغامرة: مفيش طريقة تتأكد بيها إنك مكسرتش حاجة كانت شغالة.

هندرس في الفصل ده تلات مواضيع مترابطة: Unit Testing (اختبار أصغر وحدة من الكود بمعزل عن الباقي)، Integration Testing (اختبار تفاعل عدة أجزاء مع بعض)، وMocking (محاكاة الاعتماديات الخارجية عشان تقدر تختبر بمعزل عن موارد حقيقية زي قواعد البيانات أو الشبكة).

ليه بندرس الموضوع ده دلوقتي؟ لأن كل حاجة اتعلمتها لحد دلوقتي - من Clean Code لحد Error Handling - بتاخد قيمتها الكاملة بس لما يكون فيه اختبارات بتتأكد منها. ودي نفس القاعدة اللي هنحتاجها في الفصل الجاي (Git Professional)، حيث الاختبارات هي اللي بتديك الثقة إنك تقدر تعمل Merge أو Rebase لكود جديد من غير خوف من كسر حاجة موجودة.

٧,١ Unit Testing — اختبار الوحدات

Unit Testing هو اختبار أصغر وحدة قابلة للاختبار في الكود (عادة دالة أو Method واحدة) بمعزل تام عن باقي أجزاء النظام، للتأكد إنها بتشتغل صح في كل الحالات المتوقعة، بما فيها الحالات الحدية (Edge Cases).

١. ليه Unit Testing مهم؟

من غير اختبارات، الطريقة الوحيدة للتأكد إن الكود شغال صح هي التشغيل اليدوي والفحص البصري، وده بطيء وغير دقيق وميغطيش كل الحالات الممكنة. Unit Testing بيوفر طريقة أوتوماتيكية وسريعة ومتكررة للتأكد إن كل دالة بتشتغل صح.

في مشاريع ML، Unit Testing مهم بشكل خاص لدوال معالجة البيانات والحسابات، لأن خطأ بسيط في دالة تطبيع بيانات مثلًا ممكن يفسد نتيجة التدريب كله من غير ما يظهر كخطأ واضح، وإنما كأداء أسوأ من المتوقع من غير سبب واضح.

٢. المشكلة اللي بيحلها Unit Testing

المشكلة هي الاعتماد على الفحص اليدوي للتأكد من صحة الكود، وده بطيء، وميتكررش بسهولة، وسهل تنسى تفحص حالة حدية معينة. مع مرور الوقت وتراكم التعديلات، صعب تتأكد إن تعديل جديد مكسرش سلوك قديم كان شغال صح من غير إعادة فحص كل حاجة يدويًا من الأول.

٣. إزاي كانت الشركات بتتعامل قبل تبني Unit Testing

كان شائع الاعتماد على الفحص اليدوي والتجربة المباشرة (زي تشغيل السكربت والنظر للنتيجة بالعين) للتأكد من صحة الكود، خصوصًا في المراحل المبكرة من المشروع أو في الأكواد التجريبية السريعة.

٤. ليه الطريقة القديمة مش مناسبة

الفحص اليدوي بطيء جدًا مقارنة بتشغيل اختبارات أوتوماتيكية، وميغطيش كل الحالات الممكنة بشكل منهجي، وبيعتمد على تذكر المهندس نفسه لكل الحالات اللي محتاجة فحص، وده عرضة للنسيان والخطأ البشري خصوصًا مع تكرار التعديلات على نفس الكود.

٥. الفكرة الأساسية وراء Unit Testing

لكل دالة أو وحدة كود، بتكتب اختبارات أوتوماتيكية بتستدعيها بمدخلات معروفة، وبتتأكد إن المخرجات مطابقة للمتوقع. الاختبارات دي بتتشغل تلقائيًا وبسرعة، وبتديك تأكيد فوري لو تعديل جديد كسر سلوك كان متوقع يفضل ثابت.

٦. المفاهيم الأساسية

- Arrange-Act-Assert: نمط شائع لكتابة الاختبار: تجهيز البيانات (Arrange)، تنفيذ العملية المطلوب اختبارها (Act)، التأكد من النتيجة (Assert).
- Test Coverage: نسبة الكود اللي فعليًا بيتغطى بالاختبارات، مؤشر مفيد لكن مش ضمان كافي وحده لجودة الاختبارات.
- Edge Cases: الحالات الحدية أو غير المعتادة (زي قائمة فاضية، قيمة صفر، قيمة سالبة) اللي غالبًا بتكشف أخطاء مش بتظهر في الاختبار بالحالة العادية بس.

٧. المكونات

- إطار عمل اختبار (Testing Framework) زي pytest في Python.
- دالة اختبار واحدة لكل سيناريو، باسم واضح يوضح إيه اللي بيتم اختباره.
- تأكيدات (Assertions) بتتحقق من إن النتيجة الفعلية مطابقة للنتيجة المتوقعة.

٨. إزاي المبدأ ده بيشتغل داخليًا

كل اختبار بيشغل الكود المطلوب اختباره بمدخلات محددة، وبيقارن الناتج الفعلي بالناتج المتوقع باستخدام Assertion. لو الناتج مش مطابق، الاختبار بيفشل ويوضح بالظبط أنهي جزء من الكود مش بيشتغل زي المتوقع، من غير الحاجة لتتبع المشكلة يدويًا.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في فرق الـ ML الكبرى، أي دالة معالجة بيانات أو حساب مقاييس بيكون ليها اختبارات وحدة تغطي الحالة العادية والحالات الحدية (بيانات فاضية، قيم متطرفة)، وده بيشتغل كجزء إلزامي من الـ CI Pipeline قبل قبول أي Pull Request، بحيث محدش يقدر يدمج كود بيكسر اختبار موجود من غير مراجعة صريحة.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في اختبار دوال معالجة البيانات (تطبيع، تنظيف قيم مفقودة)، اختبار دوال حساب المقاييس (دقة، Precision، Recall)، واختبار منطق التحقق من صحة البيانات (Validation) اللي اتعلمناه في الفصل السابق.

١١. امتى تكتب Unit Tests شاملة

- لأي دالة منطق أعمال أو حساب مهم متوقع يستخدم ويتعدل باستمرار.
- لأي دالة فيها حالات حدية معقدة يسهل نسيانها أو التعامل معاها بشكل خاطئ.

١٢. امتى ممكن تكتفي باختبارات محدودة

- لسكربت تجريبي بسيط جدًا ومؤقت مش متوقع يتطور لكود Production.
- لكود بسيط جدًا (زي دالة تجمع رقمين) فيه المخاطرة من الخطأ منخفضة جدًا بطبيعتها.

١٣. أفضل الممارسات

Best Practices

- اختبر الحالة العادية والحالات الحدية معًا، مش الحالة العادية بس.
- خلي كل اختبار مستقل عن باقي الاختبارات، ميعتمدش على ترتيب تنفيذهم.
- استخدم أسماء اختبار واضحة توضح بالظبط إيه السيناريو اللي بيتم اختباره.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- اختبار الحالة العادية بس من غير تغطية الحالات الحدية أو حالات الفشل المتوقعة.
- كتابة اختبارات مترابطة بتعتمد على ترتيب تنفيذها أو على حالة مشتركة بينها.
- الوثوق بنسبة Test Coverage عالية كدليل كافٍ على جودة الاختبارات، من غير التأكد الفعلي إن الاختبارات بتغطي السيناريوهات المهمة.

١٥. اعتبارات الأداء

اختبارات الوحدة المفروض تكون سريعة جدًا (أجزاء من الثانية لكل اختبار)، لأنها بتُشغل بشكل متكرر جدًا أثناء التطوير. لو اختبار وحدة بطيء، غالبًا ده مؤشر إنه فعليًا بيعتمد على مورد خارجي حقيقي (زي قاعدة بيانات) بدل ما يكون معزول تمامًا، وده موضوع Mocking اللي هنشرحه في الموضوع الجاي.

١٦. الاعتبارات الأمنية

اختبارات الوحدة بتساعد في اكتشاف حالات حدية أو مدخلات غير متوقعة ممكن تسبب ثغرات أمنية، زي التأكد إن دالة معالجة النصوص بتتعامل صح مع مدخلات طويلة جدًا أو تحتوي على رموز خاصة.

١٧. اعتبارات قابلية التوسع

وجود اختبارات وحدة شاملة بيدي ثقة للفريق إنه يقدر يعدل ويوسع الكود بأمان، لأن أي كسر في السلوك المتوقع هيظهر فورًا في الاختبارات، بدل ما يكتشف بعد وصول الكود للإنتاج.

١٨. مقارنة مع نهج بديل

البديل هو الاعتماد على الفحص اليدوي والاختبار في بيئة الإنتاج مباشرة (اللي أحيانًا بيتسمى بشكل ساخر Testing in Production). الأسلوب ده أسرع لمرة واحدة، لكنه بيعني إن أي خطأ هيكتشفه المستخدمين الحقيقيين قبل الفريق، وده مكلف جدًا في أي نظام Production حقيقي.

١٩. مثال عملي صغير

```
import pytest

def normalize(value, min_val, max_val):
    if max_val == min_val:
        raise ValueError('min_val and max_val cannot be equal')
    return (value - min_val) / (max_val - min_val)

def test_normalize_returns_expected_value():
    result = normalize(5, 0, 10)
    assert result == 0.5

def test_normalize_handles_boundary_values():
    assert normalize(0, 0, 10) == 0.0
    assert normalize(10, 0, 10) == 1.0

def test_normalize_raises_on_equal_bounds():
    with pytest.raises(ValueError):
        normalize(5, 3, 3)
```

٢٠. مثال من مشروع Production حقيقي

في مكتبة معالجة بيانات داخلية بشركة كبيرة، دالة حساب الـ Feature المشتقة (زي حساب متوسط آخر ٣٠ يوم) بيكون ليها اختبارات وحدة تغطي حالات زي بيانات فاضية، بيانات بيوم واحد بس، وبيانات فيها قيم مفقودة، عشان تضمن إن الدالة دي - اللي بتستخدمها عشرات النماذج المختلفة - بتشتغل صح في كل السيناريوهات الممكنة.

٢١. ملخص الموضوع

خلاصة Unit Testing

- اختبر أصغر وحدة كود بمعزل عن الباقي، بما فيها الحالات الحدية.
- الاختبارات السريعة والمستقلة بتشجع على تشغيلها بشكل متكرر أثناء التطوير.
- Test Coverage مؤشر مفيد، لكن مش بديل عن التفكير الفعلي في السيناريوهات المهمة.

٧,٢ Integration Testing — اختبار التكامل

Integration Testing بيختبر تفاعل عدة أجزاء من النظام مع بعض، للتأكد إنهم بيشتغلوا صح مع بعض ككل متكامل، مش بس كل جزء لوحده. ده بيغطي فجوة مهمة ميقدرش Unit Testing يغطيها بمفرده.

١. ليه Integration Testing مهم؟

حتى لو كل وحدة من وحدات النظام بتشتغل صح لوحدها (مؤكد عن طريق Unit Tests)، مفيش ضمان إنهم هيشتغلوا صح مع بعض. المشكلة ممكن تكون في الواجهة بين الوحدتين: شكل بيانات متوقع مختلف، أو ترتيب استدعاء غلط، أو افتراضات متضاربة بين الوحدتين.

في مشاريع ML، ده مهم جدًا لأن الـ Pipeline بيتكون من عدة مراحل مترابطة (تحميل، معالجة، تدريب، تقييم)، وحتى لو كل مرحلة اتختبرت لوحدها، لازم تتأكد إن المخرجات من مرحلة بتتناسب فعليًا مع المدخلات المتوقعة للمرحلة اللي بعدها.

٢. المشكلة اللي بيحلها Integration Testing

المشكلة هي الفجوة بين الوحدات المختبرة بشكل منفصل: كل دالة ممكن تعدي اختباراتها الخاصة، لكن لما تتجمع مع بعض في الـ Pipeline الكامل، ممكن يظهر تعارض في الافتراضات (زي دالة بترجع DataFrame بعمود اسمه label، ودالة تانية بتتوقع اسم target) ميظهرش إلا لما تشتغل الاتنين مع بعض فعليًا.

٣. إزاي كانت الشركات بتتعامل قبل تبني Integration Testing

كان شائع الاعتماد على Unit Tests بس، مع افتراض إن لو كل جزء شغال صح لوحده، النظام كله هيشتغل صح تلقائيًا لما يتجمع، من غير اختبار حقيقي للتكامل بين الأجزاء دي.

٤. ليه الطريقة القديمة مش مناسبة

الافتراض إن الأجزاء هتشتغل صح مع بعض تلقائيًا غير صحيح في الواقع، لأن أخطاء التكامل (زي عدم توافق شكل البيانات بين مرحلتين) بتظهر بس لما الأجزاء دي فعليًا تتفاعل مع بعض، وده اكتشاف بيحصل غالبًا متأخر جدًا، أحيانًا في بيئة الإنتاج نفسها.

٥. الفكرة الأساسية وراء Integration Testing

بتشغل عدة وحدات مع بعض فعليًا (زي تشغيل مرحلة تحميل البيانات ثم تمرير الناتج مباشرة لمرحلة المعالجة)، وبتتأكد إن التدفق الكامل بينهم بيشتغل صح من غير أخطاء أو تعارضات في الافتراضات.

٦. المفاهيم الأساسية

- Integration Point: أي نقطة تفاعل بين وحدتين أو أكتر، وهي المصدر الأساسي لأخطاء التكامل.
- Test Environment: بيئة اختبار مشابهة للإنتاج (لكن معزولة) تسمح باختبار التكامل بأمان من غير التأثير على بيانات حقيقية.
- Contract Testing: نوع متخصص من اختبار التكامل بيتأكد إن الواجهة بين وحدتين (شكل المدخلات والمخرجات) متوافقة، حتى لو الوحدتين بيتطوروا بشكل منفصل.

٧. المكونات

- سيناريو اختبار بيشغل عدة وحدات مع بعض بترتيب واقعي.
- بيئة اختبار معزولة (زي قاعدة بيانات اختبار منفصلة) تحاكي بيئة الإنتاج من غير المخاطرة الحقيقية.
- تأكيدات (Assertions) بتتحقق من صحة التدفق الكامل، مش بس صحة كل خطوة لوحدها.

٨. إزاي المبدأ ده بيشتغل داخليًا

الاختبار بيشغل تسلسل حقيقي من العمليات المترابطة (زي تحميل بيانات حقيقية من ملف اختبار، ثم معالجتها، ثم تمريرها لدالة التدريب)، ويتأكد إن الناتج النهائي للسلسلة دي صحيح. أي عدم توافق بين مرحلتين هيظهر كفشل في الاختبار، حتى لو كل مرحلة لوحدها عدت اختبارات الوحدة الخاصة بيها بنجاح.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في منصات ML الكبرى، بيتم تشغيل اختبارات تكامل بتحاكي الـ Pipeline الكامل من البداية للنهاية (End-to-End) على بيانات اختبار صغيرة لكن واقعية، كجزء من الـ CI Pipeline، قبل ما أي تغيير كبير في بنية الـ Pipeline يتم قبوله ودمجه.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في اختبار التدفق الكامل من تحميل البيانات لحد إنتاج نموذج مدرب، اختبار التكامل بين النموذج المدرب وخدمة الـ Serving اللي بتقدمه للمستخدمين، واختبار التكامل بين الـ Pipeline ومصادر البيانات الخارجية الحقيقية.

١١. امتى تكتب Integration Tests

- لما عندك تدفق مهم بيمر بعدة مراحل أو وحدات مترابطة.
- لما تعديل في وحدة معينة ممكن يأثر على وحدات تانية بشكل غير مباشر وصعب توقعه من غير اختبار فعلي.

١٢. امتى Integration Testing يكون زيادة عن اللزوم

- لوحدات بسيطة جدًا ومعزولة تمامًا مافيهاش أي تكامل حقيقي مع وحدات تانية.
- لما الـ Unit Tests الشاملة كافية لتغطية كل السيناريوهات المهمة من غير الحاجة لاختبار تكامل إضافي.

١٣. أفضل الممارسات

Best Practices

- ركز اختبارات التكامل على نقاط التفاعل المهمة والحرجة، مش كل تفاعل ممكن بين كل وحدتين.
- استخدم بيئة اختبار معزولة تحاكي الإنتاج، متختبرش على بيانات إنتاج حقيقية مباشرة.
- خلي اختبارات التكامل مكملة للـ Unit Tests، مش بديل عنها.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- الاعتماد على Unit Tests بس والافتراض إن التكامل هيشتغل صح تلقائيًا.
- اختبار التكامل على بيانات أو بيئة إنتاج حقيقية، وده بيحمل مخاطرة غير ضرورية.
- كتابة اختبارات تكامل بطيئة جدًا وغير مستقرة (Flaky) لدرجة إن الفريق بيبدأ يتجاهلها أو يعطلها.

١٥. اعتبارات الأداء

اختبارات التكامل عادة أبطأ من اختبارات الوحدة، لأنها بتشغل عمليات فعلية متعددة مع بعض. من المقبول إنها تكون أبطأ، لكن لازم توازن بين شمولية الاختبار وسرعة تنفيذه، عن طريق التركيز على السيناريوهات الأهم بدل اختبار كل مسار ممكن.

١٦. الاعتبارات الأمنية

اختبارات التكامل بتساعد في اكتشاف مشاكل أمنية بتظهر بس في التفاعل بين وحدات متعددة، زي تسريب بيانات حساسة بين مرحلتين بسبب افتراض خاطئ حول تنسيق البيانات المتبادلة بينهم.

١٧. اعتبارات قابلية التوسع

وجود اختبارات تكامل شاملة بيدي ثقة للفريق إنه يقدر يعدل بنية الـ Pipeline أو يضيف مراحل جديدة، مع التأكد إن التكامل بين كل الأجزاء لسه شغال صح، حتى مع نمو تعقيد النظام.

١٨. مقارنة مع نهج بديل

البديل هو الاعتماد على Unit Tests بس من غير اختبار تكامل حقيقي، والاكتشاف اليدوي لأي مشاكل تكامل لما تظهر في الإنتاج. الأسلوب ده أسرع في كتابة الاختبارات في البداية، لكنه بيؤخر اكتشاف مشاكل التكامل لمرحلة متأخرة ومكلفة جدًا. الشركات الكبرى بتستثمر في الاتنين معًا كطبقتين مكملتين من الحماية.

١٩. مثال عملي صغير

```
def test_data_pipeline_end_to_end(tmp_path):
    # Arrange - create a small realistic test file
    test_file = tmp_path / 'test_data.csv'
    test_file.write_text('age,income,label\n25,50000,1\n40,80000,0\n')

    # Act - run the actual pipeline stages together
    raw_data = load_training_data(str(test_file))
    validate_training_data(raw_data)
    processed_data = preprocess_features(raw_data)
    model = train_model(processed_data)

    # Assert - the full flow produces a usable, trained model
    assert model is not None
    predictions = model.predict(processed_data[['age', 'income']])
    assert len(predictions) == len(raw_data)
```

٢٠. مثال من مشروع Production حقيقي

في منصة ML كبيرة، اختبار تكامل بيشغل الـ Pipeline الكامل من تحميل بيانات اختبار صغيرة، مرورًا بكل مراحل المعالجة، وصولًا لنشر نموذج تجريبي، ويتأكد إن النموذج ده يقدر فعليًا يستقبل طلب تنبؤ ويرجع نتيجة صحيحة، قبل ما أي تحديث كبير في بنية الـ Pipeline يتم اعتماده.

٢١. ملخص الموضوع

خلاصة Integration Testing

- اختبر تفاعل عدة وحدات مع بعض، مش بس كل وحدة لوحدها.
- ركز على نقاط التفاعل الحرجة والمهمة، واستخدم بيئة اختبار معزولة.
- Integration Testing مكمل لـ Unit Testing، مش بديل عنه.

٧,٣ Mocking — محاكاة الاعتماديات

Mocking هو تقنية لاستبدال اعتماد حقيقي (زي اتصال بقاعدة بيانات أو طلب شبكة) بنسخة وهمية (Mock) بسيطة ومتحكم فيها بالكامل، بحيث تقدر تختبر الكود بمعزل عن الموارد الخارجية الحقيقية.

١. ليه Mocking مهم؟

لو الكود بيعتمد على مورد خارجي حقيقي (قاعدة بيانات، خدمة شبكة، نظام ملفات)، اختباره مباشرة بيبقى بطيء (اتصال شبكة فعلي)، وغير مستقر (الخدمة الخارجية ممكن تكون غير متاحة وقت الاختبار)، وأحيانًا خطير (تعديل بيانات حقيقية بالغلط أثناء الاختبار).

في مشاريع ML، ده بيظهر باستمرار: اختبار كود بيتفاعل مع خدمة تخزين سحابية أو Feature Store حقيقي مش عملي في كل مرة تشغل فيها الاختبارات، فبتحتاج طريقة لمحاكاة التفاعل ده بأمان وسرعة.

٢. المشكلة اللي بيحلها Mocking

المشكلة هي الاختبارات اللي بتعتمد على موارد خارجية حقيقية، وده بيخليها بطيئة، وغير مستقرة (Flaky) لأن نتيجتها ممكن تتأثر بعوامل خارجة عن الكود نفسه (زي بطء الشبكة أو توقف خدمة معينة مؤقتًا)، وأحيانًا خطيرة لو الاختبار بيعدل بيانات حقيقية بالغلط.

٣. إزاي كانت الشركات بتتعامل قبل تبني Mocking

كان شائع تشغيل الاختبارات مباشرة ضد موارد حقيقية (زي قاعدة بيانات اختبار مشتركة)، وده كان بيسبب مشاكل زي تعارض بين اختبارات مختلفة بتشتغل في نفس الوقت على نفس المورد، أو بطء شديد في تشغيل مجموعة الاختبارات كاملة.

٤. ليه الطريقة القديمة مش مناسبة

الاعتماد على موارد حقيقية بيخلي الاختبارات بطيئة جدًا مع نمو عددها، وغير قابلة للتشغيل المتوازي بسهولة بسبب تعارض الوصول لنفس المورد، وغير مستقرة لأن نتيجتها بتعتمد جزئيًا على حالة المورد الخارجي وقت التشغيل، مش بس على صحة الكود نفسه.

٥. الفكرة الأساسية وراء Mocking

بدل الاعتماد الحقيقي، بتستبدله بنسخة وهمية (Mock) بتتحكم فيها بالكامل: بتحدد بالظبط إيه اللي المفروض ترجعه لما تتنادى بطريقة معينة، وده بيسمح لك تختبر منطق الكود نفسه بمعزل تام عن أي عامل خارجي، وده نفس فكرة Dependency Injection اللي اتعلمناها في فصل SOLID: لو الكود بيستقبل اعتماده من برا، تقدر تحقن نسخة وهمية بسهولة وقت الاختبار.

٦. المفاهيم الأساسية

- Mock: كائن وهمي بيحاكي سلوك اعتماد حقيقي، وبيتحكم المختبر فيه بالكامل.
- Stub: نوع مبسط من الـ Mock بيرجع قيمة ثابتة محددة مسبقًا من غير أي منطق إضافي.
- Spy: نوع من الـ Mock بيسجل إزاي اتنادى (كام مرة، بإيه معطيات) عشان تتأكد إن الكود استخدمه بشكل صحيح.

٧. المكونات

- الاعتماد الحقيقي (زي اتصال بقاعدة بيانات) اللي عايز تستبدله وقت الاختبار.
- مكتبة Mocking (زي unittest.mock في Python) بتوفر أدوات لإنشاء الكائنات الوهمية.
- تحديد السلوك المتوقع من الـ Mock (إيه اللي المفروض يرجعه لما يتنادى بطريقة معينة).

٨. إزاي المبدأ ده بيشتغل داخليًا

بدل ما الكود يتفاعل مع الاعتماد الحقيقي، بيتفاعل مع كائن Mock بيتصرف بنفس الواجهة (Interface) لكن بسلوك محدد ومتحكم فيه بالكامل من قبل الاختبار نفسه. ده بيسمح باختبار منطق الكود بشكل معزول تمامًا، من غير أي تأثير من عوامل خارجية حقيقية.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في منصات ML الكبرى، اختبارات الوحدة لأي كود بيتفاعل مع خدمات خارجية (قواعد بيانات، Feature Stores، خدمات تخزين سحابية) بتستخدم نسخ وهمية بدل الاتصال الحقيقي، وده بيخلي مجموعة الاختبارات كاملة تشتغل بسرعة كبيرة (ثوانٍ بدل دقائق) كجزء من الـ CI Pipeline في كل Pull Request.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في اختبار كود بيتفاعل مع قواعد بيانات أو Feature Stores حقيقية، اختبار كود بيرسل إشعارات أو يستدعي APIs خارجية، واختبار كود بيعتمد على توقيت النظام (زي دوال بتستخدم الوقت الحالي) عن طريق محاكاة قيمة وقت ثابتة.

١١. امتى تستخدم Mocking

- لما الكود بيعتمد على مورد خارجي بطيء أو غير موثوق (شبكة، قاعدة بيانات، خدمة خارجية).
- لما عايز تختبر حالات صعب تحققها بمورد حقيقي (زي فشل الاتصال بالشبكة تحديدًا).

١٢. امتى Mocking يكون مبالغ فيه

- لما بتموك كل حاجة حتى المنطق الداخلي البسيط اللي مافيهوش اعتماد خارجي حقيقي، وده بيخلي الاختبار مايختبرش حاجة فعلية.
- في اختبارات التكامل اللي هدفها بالتحديد التأكد من التفاعل الحقيقي بين الأجزاء، فيه الإفراط في الـ Mocking بيلغي الغرض من الاختبار نفسه.

١٣. أفضل الممارسات

Best Practices

- استخدم Mocking للاعتماديات الخارجية بس (شبكة، قاعدة بيانات)، مش للمنطق الداخلي البسيط.
- تأكد إن الـ Mock بيحاكي الواجهة الحقيقية بدقة، عشان الاختبار يفضل ذا معنى وممثل للواقع.
- وازن بين Unit Tests بـ Mocking (سريعة ومعزولة) وIntegration Tests بموارد حقيقية أو شبه حقيقية (أبطأ لكن أشمل).

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- الإفراط في استخدام Mocking لدرجة إن الاختبار مايعبرش عن أي سلوك حقيقي للنظام.
- Mock بيحاكي واجهة قديمة أو غير دقيقة للاعتماد الحقيقي، وده بيخلي الاختبار ينجح رغم إن الكود ممكن يفشل فعليًا مع الاعتماد الحقيقي.
- الاعتماد على Mocking بس من غير أي اختبار تكامل حقيقي يتأكد من التوافق الفعلي مع الاعتماد الحقيقي من وقت لآخر.

١٥. اعتبارات الأداء

استخدام Mocks بدل الموارد الحقيقية بيسرع الاختبارات بشكل كبير جدًا، من دقايق لثواني في كتير من الحالات، وده بيشجع الفريق على تشغيل الاختبارات بشكل متكرر أثناء التطوير من غير إحباط من بطء الانتظار.

١٦. الاعتبارات الأمنية

استخدام Mocks بدل الاتصال بموارد إنتاج حقيقية بيمنع أي خطر لتسريب أو تعديل بيانات حساسة بالغلط أثناء تشغيل الاختبارات، وده بيوفر طبقة حماية إضافية مهمة، خصوصًا في بيئات CI المشتركة.

١٧. اعتبارات قابلية التوسع

الاختبارات السريعة بفضل Mocking بتسمح للفريق يضيف اختبارات جديدة باستمرار من غير القلق من زيادة وقت تشغيل مجموعة الاختبارات كاملة بشكل غير محتمل مع نمو المشروع.

١٨. مقارنة مع نهج بديل

البديل هو استخدام موارد حقيقية (أو شبه حقيقية زي قاعدة بيانات اختبار مخصصة) في كل الاختبارات من غير Mocking. الأسلوب ده بيوفر ثقة أعلى في التوافق الحقيقي، لكنه أبطأ وأصعب في الإدارة. التوازن الأمثل المستخدم في الشركات الكبرى هو استخدام Mocking في اختبارات الوحدة الكتيرة والسريعة، واستخدام موارد حقيقية أو شبه حقيقية في عدد أقل من اختبارات التكامل الشاملة.

١٩. مثال عملي صغير

```
from unittest.mock import Mock

class ModelRegistry:
    def save(self, model, name):
        ...  # Real implementation talks to a cloud storage service

class ModelTrainer:
    def __init__(self, registry: ModelRegistry):
        self.registry = registry

    def train_and_save(self, data):
        model = fit_model(data)
        self.registry.save(model, name='latest')
        return model

def test_train_and_save_calls_registry_correctly():
    mock_registry = Mock()
    trainer = ModelTrainer(registry=mock_registry)

    trainer.train_and_save(sample_data)

    mock_registry.save.assert_called_once()
    args, kwargs = mock_registry.save.call_args
    assert kwargs['name'] == 'latest'
```

٢٠. مثال من مشروع Production حقيقي

في نظام ML بيشتغل على منصة سحابية، اختبارات الوحدة لكود التدريب بتستخدم نسخة وهمية من خدمة تخزين النماذج السحابية بدل الاتصال الحقيقي، وده بيسمح للفريق يشغل آلاف الاختبارات في دقايق قليلة كجزء من الـ CI Pipeline، بدل ساعات لو كانت كل الاختبارات بتتصل بالخدمة الحقيقية فعليًا.

٢١. ملخص الموضوع

خلاصة Mocking

- استبدل الاعتماديات الخارجية بنسخ وهمية متحكم فيها بالكامل وقت الاختبار.
- استخدمه للموارد الخارجية بس، مش للمنطق الداخلي البسيط.
- وازن دايمًا بين اختبارات سريعة بـ Mocking واختبارات تكامل أشمل بموارد حقيقية.

مشروع عملي على الفصل: بناء مجموعة اختبارات متكاملة لـ Mini ML Pipeline

المشروع ده هدفه إنك تدمج الثلاث مستويات من الاختبار (Unit Testing، Integration Testing، Mocking) في مجموعة اختبارات شاملة لـ Pipeline تدريب بسيط.

السيناريو: عندك Pipeline بيحمل بيانات، يعالجها، يدربها، ويحفظ النموذج في خدمة تخزين سحابية. عايز تبني مجموعة اختبارات تدي ثقة كاملة إن الـ Pipeline ده شغال صح، من غير ما تعتمد على الاتصال الفعلي بالخدمة السحابية في كل مرة.

خطوات التنفيذ

- اكتب Unit Tests لكل دالة معالجة بيانات، تغطي الحالة العادية والحالات الحدية (بيانات فاضية، قيم متطرفة، قيم مفقودة).
- اكتب Unit Tests لدالة حساب المقاييس، تتأكد من صحة الحسابات في حالات معروفة النتيجة مسبقًا.
- استخدم Mocking لاستبدال خدمة التخزين السحابي بنسخة وهمية، واكتب اختبار يتأكد إن الكود بينادي على دالة الحفظ بالمعطيات الصحيحة.
- اكتب Integration Test واحد شامل بيشغل الـ Pipeline كامل (تحميل، معالجة، تدريب) على بيانات اختبار صغيرة وواقعية، ويتأكد إن النموذج الناتج قابل للاستخدام فعليًا.
- شغل مجموعة الاختبارات كاملة وتأكد إنها بتاخد ثواني بس، مش دقايق، بفضل استخدام Mocking للاعتماديات الخارجية.

تحديات شائعة عند التطبيق

- معرفة الحد الفاصل بين Unit وIntegration Testing: القاعدة العملية هي إن Unit Test بيختبر دالة واحدة بمعزل تام، بينما Integration Test بيختبر تفاعل حقيقي بين عدة دوال أو وحدات مع بعض. لو مش متأكد، اسأل نفسك: هل بختبر منطق دالة واحدة، ولا تفاعل عدة أجزاء؟
- الإفراط في Mocking لدرجة فقدان معنى الاختبار: لو موكيت كل حاجة حتى المنطق الأساسي اللي بتحاول تختبره فعليًا، الاختبار مابقاش بيتأكد من حاجة حقيقية. استخدم Mocking للاعتماديات الخارجية بس.
- تجاهل صيانة الاختبارات مع تطور الكود: الاختبارات مش كتابة مرة واحدة وخلاص، لازم تتحدث مع أي تغيير مقصود في السلوك، وإلا هتبدأ تفشل بشكل مزعج أو الأسوأ، هتتعطل بالكامل من الفريق بسبب الإحباط.

إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي

الاختبار الشامل بيظهر في كل جزء من نظام ML ناضج: Unit Tests بتتأكد من صحة كل دالة معالجة بيانات وحساب مقاييس، Integration Tests بتتأكد إن الـ Pipeline الكامل شغال صح من البداية للنهاية، وMocking بيخلي الاختبارات دي سريعة وآمنة من غير الاعتماد على موارد إنتاج حقيقية.

في الشركات الكبرى، أي تغيير في كود ML لازم يعدي مجموعة اختبارات شاملة قبل الدمج، وده اللي بيدي الفريق الثقة إنه يقدر يحسن ويطور النظام باستمرار من غير خوف من كسر سلوك كان شغال صح من قبل - وهي بالظبط الثقة اللي هنحتاجها بشكل مباشر في الفصل الجاي لما نتكلم عن Merge وRebase في Git.

ملخص الفصل السابع وربطه بالفصل القادم

في الفصل ده اتعلمت الثلاث مستويات الأساسية من الاختبار: Unit Testing لاختبار أصغر وحدة كود بمعزل عن الباقي، Integration Testing لاختبار تفاعل عدة أجزاء مع بعض، وMocking لمحاكاة الاعتماديات الخارجية بحيث الاختبارات تفضل سريعة وآمنة ومستقلة عن الموارد الحقيقية.

الفصل الجاي والأخير (Git Professional) هيبني على الثقة اللي اتعلمناها هنا. هنشرح إزاي تدير الكود بشكل احترافي عبر فريق كامل: استراتيجيات الفروع (Branch Strategy)، Pull Requests وCode Review، وعمليات زي Merge وRebase. كل العمليات دي بتعتمد بشكل أساسي على وجود اختبارات قوية تدي الفريق الثقة إن أي تغيير جديد (حتى لو جاي من مهندس تاني) مكسرش حاجة كانت شغالة قبل كده.